



RollArt: 通过分离式基础设施实现智能体强化学习训练的缩放

Wei Gao^{†*}, Yuheng Zhao^{†*}, Tianyuan Wu^{†*}, Shaopan Xiong^{‡*}, Weixun Wang^{‡*}, Dakai An[†],
Lunxi Cao[†], Dilxat Muhtar[‡], Zichen Liu[‡], Haizhou Zhao[‡], Ju Huang[‡], Siran Yang[‡],
Yongbin Li[¶], Wenbo Su[‡], Jiamang Wang[‡], Lin Qu[‡], Bo Zheng[‡], Wei Wang[†]

[†]HKUST [‡]Alibaba Group [¶]Tongyi Lab, Alibaba

摘要

Agentic 强化学习 (RL) 使大模型 (LLMs) 能够实现自主决策与长期规划。与标准的大模型后训练不同, Agentic RL 工作负载具有高度的异构性, 结合了计算密集型的预填充阶段、带宽受限的解码过程以及状态敏感、CPU 密集的环境仿真。我们认为, 高效的 Agentic RL 训练需要解耦基础设施, 以利用专用且最适配的硬件。然而, 简单的解耦会因各阶段间复杂的依赖关系引入显著的同步开销和资源利用率低下问题。

我们提出 RollArt, 一个专为解耦基础设施上的多任务智能体强化学习设计的分布式系统, 旨在最大化吞吐量。RollArt 基于三大核心原则: (1) 硬件亲和性工作负载映射, 将计算密集型和带宽密集型任务路由至最匹配的 GPU 设备; (2) 细粒度异步性, 在轨迹级别管理执行以缓解资源空洞问题; (3) 状态感知计算, 将无状态组件 (如奖励模型) 卸载至无服务器基础设施, 实现弹性扩展。实验结果表明, RollArt 有效提升了训练吞吐量, 并相比单体式和同步基准实现了 1.35-2.05 $\times$ 的端到端训练时间降低。我们还在阿里集群上使用超过 3,000 块 GPU 对数百亿参数的 MoE 模型进行 Qoder 产品训练, 进一步验证了 RollArt 的可扩展性和鲁棒性。代码已开源, 地址见 <https://github.com/alibaba/ROLL>。

1 引言

强化学习 (RL) 已成为推动大模型 (LLMs) 从被动逻辑推理迈向自主决策与长时程规划的核心方法 [1, 9, 44]。这一范式被称为 Agentic RL, 要求大模型在复杂、动态的环境中有效运作, 涵盖工具使用 [18, 61]、网页导

航 [25, 27, 54] 以及通用计算机控制 [32, 34, 37]。与传统的后训练大模型不同, Agentic RL 强调通过大量实践进行学习: 智能体主动与外部环境交互, 生成序列、多轮次的轨迹, 从而通过试错解决复杂的推理任务。

Agentic RL 训练流水线以迭代循环的形式运行, 包含三个阶段: rollout、reward 和 training。在 rollout 阶段, 智能体 LLM 与环境进行多轮反馈环, 生成动作 token 并接收观测, 直到满足终止条件, 这构成了一个轨迹。这些轨迹随后在奖励阶段被评估, 通常由分离的模型或代码沙箱完成, 以分配标量奖励信号。最后, 在训练阶段, 这些标注的轨迹被用来更新智能体 LLM 的权重。这些更新后的参数随后同步回 rollout 工作节点, 用于下一轮迭代, 从而形成数据生成与策略更新的闭环。

随着研究机构深入探索 Agentic RL 的缩放法则 [10, 58], 一个基础的基础设施挑战浮现: 流水线中的资源需求具有高度的异构性, 且常常存在冲突。仅 rollout 阶段就呈现出复杂的复合型工作负载。大语言模型 (LLM) 生成在 TFLOPS 高耗能的预填充阶段——最适合由计算优化硬件 (例如 NVIDIA H800) 支持——与带宽受限的解码阶段之间交替进行, 后者则受益于高内存带宽 (例如 H20)。这些阶段的比例由任务结构决定: 长时域任务如 FrozenLake [11] 或 SWE-bench [26] 以预填充为主, 而短回合任务如 GEM-math/GEM-game [5] 则以解码为主。与此同时, 智能体 LLM 必须与数千个并行环境交互。这些环境通常是有状态的、CPU 密集型仿真, 若与 GPU 节点共置, 将引发严重的资源争用。相比之下, 奖励阶段通常由无状态、可并行化的工作进程组成, 执行从简单的 CPU 脚本 [15, 21] 到复杂的基于 GPU 的模型评判 [53, 70] 等多种评估。最后, 训练阶段需要高端 GPU 及高速互连, 以维持大规模参数最优化

*高伟、赵宇恒、吴天源、熊绍盼和王维勋对本研究贡献相等。

所需的高 TFLOPS。

我们认为,单一的庞大簇无法高效满足多任务 Agentic 强化学习训练中多样且相互冲突的资源需求。虽然一个自然的解决方案是利用解耦架构,将各个阶段路由至最适配的硬件,但当前系统未能充分实现这一潜力。行业标准系统如 veRL [50,51]、slime [72] 和 rLLM [57] 仍受限于单体 GPU 簇。较新的框架如 AWorld [65] 和 DeepSWE [35] 通过将环境卸载至 Kubernetes [4] 簇引入了部分解耦。然而,它们仍然将资源密集型的轨迹生成与训练阶段共置,导致资源不匹配问题未被解决。即使最近的一些系统尝试将训练与轨迹生成解耦,例如 StreamRL [69]、AsyncFlow [17]、SeamlessFlow [59] 和 Laminar [49],也采用粗粒度方法。这些系统大多忽视了 Agentic 任务与环境固有的异构性,限制了其充分利用专用硬件加速轨迹生成的能力。

虽然拆分能够实现硬件特化,但其引入了严重的编排挑战,主要源于异构通信模式和同步开销。首先,数据传输需求高度不平衡。训练簇与回放簇之间的权重同步对带宽要求极高,涉及数百吉字节的大规模数据传输,延迟可达数十秒量级。相反,智能体大语言模型 (LLM) 与环境之间的轨迹传输数据量较小 (千字节至吉字节),但存在极端方差:环境不稳定可能引发长尾延迟,达到数百秒 (Figure 5a),严重阻塞后续阶段。其次,在同步训练中,这些通信延迟会形成“资源气泡” (Figure 2-左),导致昂贵的训练 GPU 在等待拖慢的环境或权重更新时处于空闲状态。尽管可以尝试将现有的分散式强化学习框架 [17,59,69] 与由 Kubernetes 编排的智能体环境进行拼接,但这种临时集成仅适用于单一任务、配备专用高速网络的场景,无法扩展至生产级多租户集群,因为跨簇链路通常表现出动态且异构的带宽特性。

为应对上述挑战,我们提出 RollArt,一个分布式系统,旨在最大化异构基础设施上多任务智能体强化学习的吞吐量。RollArt 源自我们此前关于 ROLL [60] 与 ROLL-Flash [33] 的技术报告。与将集群视为统一资源池的系统不同,RollArt 根据三个核心设计原则编排强化学习流水线。(1) **硬件亲和性工作负载映射**: RollArt 允许用户将整个阶段绑定至专用硬件资源池 (例如,将训练限制在计算优化型 GPU 上)。它还支持阶段内的细粒度亲和性。例如,可将预填充密集、计算密集型任务 (如 FrozenLake 和 SWE-bench) 路由至 H800 GPU,而将解码主导、带宽受限的任务 (如 GEM-math) 调度至 H20 GPU。硬件亲和性可充分挖掘可用资源,最大化系

统吞吐量。(2) **细粒度异步性**: 在采样过程中,该原则主张以轨迹级别而非批量级别进行操作。通过交错执行环境交互、大语言模型生成与奖励计算,系统有效掩盖跨集群通信延迟,并防止较慢的环境节点阻塞整个流水线。对于异步训练,需对采样与训练之间的异步程度进行细粒度控制,以平衡梯度稳定性与系统吞吐量。(3) **状态感知计算**: 为优化资源效率,RollArt 显式根据组件的状态需求进行分类。它将固有无状态组件 (如奖励模型) 卸载至无服务器基础设施 (如 Function Compute [3]),使系统实现零开销自动伸缩、多租户支持与容错能力,且无需专用热备 GPU。

我们在 RollArt 中实现了这些原则,这是一个分布式系统,将声明式编程模型与感知异构性的运行时相结合。通过基于 Python 的装饰器,用户显式定义特定子任务的硬件亲和性,并将无状态组件注册到外部无服务器接口。在底层,RollArt 的资源管理器将异构资源分配给专用的工作者,而分布式运行时则协调异步的、轨迹级的工作流。为最大化硬件效率,运行时集成了优化的执行引擎,使用推理引擎 [2,28,45] 实现高通量生成,使用训练引擎 [52] 支持分布式训练。运行时还提供了多种传输通道,利用潜在的异构网络结构 (NVLink、InfiniBand、以太网),优化了稳定性关键的轨迹传输以及带宽密集型的权重同步。

我们从头开始实现了约 6 万行的 Python 代码的 RollArt。通过在分离的簇上训练多个 Qwen 模型 [8],我们评估了 RollArt,结果表明其相较于单体和同步基准,最高可实现 $2.05\times$ 的加速效果。为验证在生产环境中的可扩展性和稳定性,我们将 RollArt 部署用于训练 Qoder [42] 产品的大规模专家混合 (MoE) 语言模型。该部署在超过 3,000 个 GPU 的集群上连续运行一周,证实了 RollArt 在大规模场景下能够维持高通量和强大的容错能力。

2 背景

2.1 Agentic 强化学习训练

训练流水线。多任务智能体强化学习的训练流水线是一个包含三个不同计算阶段的迭代环。第一阶段是用于经验收集的 rollout。在此阶段,智能体大语言模型 (演员) 与并行的环境交互以生成训练数据。与标准的大语言模型推理不同,该过程是多轮且有状态的 [10,58]。在每一轮中,智能体观察状态,生成动作 token 序列,并将其

表 1: 采用的 Agentic 环境分类

Environment	Task Domain	Modality	#Turns
SWE-bench [26]	SWE	Text	30–50
WebShop [63]	Web	Text	5–30
FrozenLake [11]	Game	Text, Visual	20–100
GEM-math [5]	Math+Tool Use	Text	< 5
GEM-game [5]	Game	Text	1

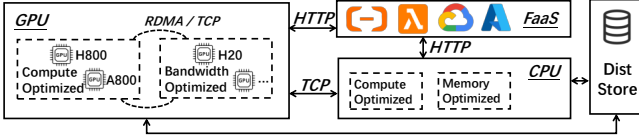


图 1: 用于 Agentic 强化学习训练的解耦基础设施。

提交给环境。环境执行该动作（例如，运行代码、点击链接），并返回反馈。此循环持续进行，直到满足终止条件，从而产生一系列状态-动作对，称为轨迹。一旦轨迹完成，系统进入**奖励阶段**，由一个奖励工作者评估智能体动作的质量。该评估产生一个标量奖励信号，可通过从轻量级基于规则的检查 [21] 到计算密集型的基于模型的判断（例如，大语言模型作为裁判 [53, 70]）等多种方法计算得出。最后，收集到的轨迹和奖励被**训练阶段**使用，通过强化学习算法（例如，PPO [43]，GRPO [48]）更新智能体大语言模型的权重。为了提升训练性能 [12]，强化学习研究人员通常采用同步强化学习训练，这要求在每个步骤中，滚动生成阶段与训练阶段之间的模型权重保持严格同步。

Environment Heterogeneity Agentic 强化学习中的一个关键挑战是环境的极端多样性，这决定了系统的计算特性。如 Table 1 所述，Agentic 任务在模态和交互频率上存在显著差异。诸如 SWE-bench [26]（软件工程）、FrozenLake [11]（视觉游戏）和 WebShop [63]（电子商务）等复杂推理任务需要较长的交互时域（最多可达 30 – 100 轮）。频繁交互迫使智能体大语言模型反复处理不断增长的上下文历史，导致工作负载以预填充为主，计算开销巨大。相反，GEM-Math 和 GEM-Game [5] 等任务可能交互轮次较少（最多五轮），但每个动作需要生成更长的思维链。这类工作负载以解码为主，使瓶颈从计算转移到内存带宽。这种方差意味着基础设施必须根据当前任务执行过程的具体特征进行动态调整。

表 2: NVIDIA 显卡规格。

Hardware Specification	H20	H800
TFLOPS	148	989.5
HBM capacity	96GB	80GB
HBM bandwidth	4TB/s	3.35TB/s
NVLink bandwidth	900GB/s	400GB/s
Normalized Cost [71]	1.00	2.85

2.2 解聚簇基础设施

解耦的必要性。智能体强化学习工作负载的极端异构性，从计算密集型训练到有状态环境仿真 (§2.1)，使得单一架构效率低下。因此，智能体强化学习训练必须转向一种**解耦式基础设施**，将这些需求冲突的计算阶段解耦为专用资源池。如 Figure 1 所示，在典型的解耦式基础设施中，训练簇利用高端、计算优化的 GPU（例如 NVIDIA H800）实现大规模吞吐；推理簇采用带宽优化的硬件（例如 H20）以支持内存受限的解码任务；CPU 簇提供弹性容量，用于由 Kubernetes [4] 协调的多样化容器化运行时环境；而**无服务器基础设施**则处理突发性、无状态的工作负载，如奖励评估。这些资源池通过标准网络结构互联，依赖分布式存储实现持久化日志记录和容错。

Sync. vs. Async. Training. 虽然分解能够解决资源不匹配问题，但它给训练范式引入了非平凡的编排挑战，该范式决定了系统吞吐量与算法一致性之间的权衡。

1) **同步训练**：此范式通过在接收来自训练簇的最新模型权重之前阻塞回放阶段，强制实现严格的一致性。然而，在解耦情景中，这会引入大量的“依赖气泡”（Figure 2-左），即在高延迟的权重同步和拖后腿的环境步骤期间，昂贵的 GPU 处于空闲状态 (§3)。

2) **异步训练**：为缓解这些延迟，系统可采用异步范式（例如，一次性强化学习训练 [36]，如 Figure 2-右所示）。在此模式下，数据收集与训练并行执行：训练阶段使用稍旧策略生成的轨迹（例如，在一次性训练中落后一迭代），而数据收集阶段则持续生成新数据。该方法有效掩盖了去耦合架构固有的同步延迟和慢节点效应，以一定程度的策略新鲜度损失（陈旧性）为代价，换取最大化的硬件利用率和系统吞吐量。

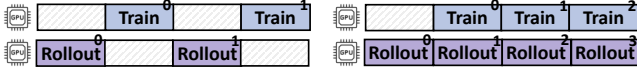


图 2: 同步训练与异步训练。

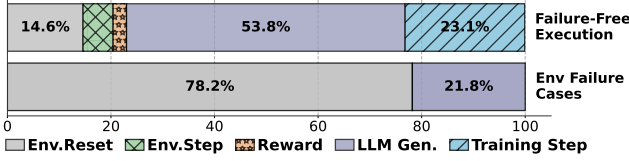


图 3: 训练步骤的分解: 成功运行 (上方, 平均耗时 365.7 秒) 与环境故障导致的执行失败 (下方, 平均耗时 513.3 秒)。

3 特性描述与系统需求

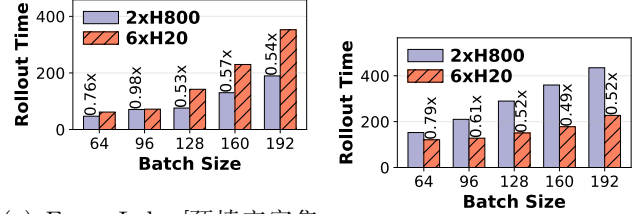
为了激发 RollArt 的设计思路, 我们对多任务 Agentic 强化学习训练进行了全面的工作负载特征分析。基于这些经验观察, 我们推导出的一组关键的系统需求 (如下框所示), RollArt 必须满足这些需求。

3.1 阶段计算

训练步骤延迟分解。 我们首先对标准训练迭代的端到端延迟进行分析, 以识别主要的成本组件。我们在 32 块 H800 GPU 上使用 SWE-bench 环境 (批量大小 128) 训练 Qwen3-8B/32K, 其中智能体大模型与容器化沙箱环境交互, 完成软件工程任务。该交互包含两个核心操作: 通过 Docker 镜像拉取和容器启动完成环境初始化的 env.reset, 以及执行智能体动作的 env.step。

Figure 3 将五次成功迭代的延迟与包含环境失败的五次迭代进行了对比。在成功运行中, 平均每次迭代耗时为 366 秒, 其中大语言模型 (LLM) 生成占据主要部分 (54%), 其次是训练 (23%) 和环境初始化 (15%)。然而, 在因环境故障导致超时的迭代中, 平均时间飙升至 513 秒。关键的是, 在这些失败场景下, env.reset 单独就消耗了滚动执行时间的 78%, 导致性能瓶颈完全从 GPU 计算转移到环境开销。我们的生产数据显示, 此类失败并非罕见的边缘情况, 大约每十次迭代就会发生一次。尽管先前的研究 [14, 20] 报告称 LLM 生成主导强化学习 (RL) 运行时间 ($>80\%$), 但我们的实证研究揭示, 在 Agentic RL 中, 高效管理环境生命周期才是最关键的挑战。

Divergent Hardware Affinities in Generation. 现代 GPU



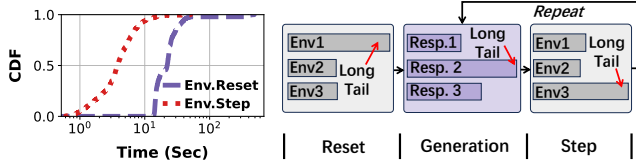
(a) FrozenLake [预填充密集型]

(b) GEM-Math [解码密集型]。

图 4: 不同任务在 H20 和 H800 GPU 上, 不同批量大小下的端到端部署时间 (秒)。

在计算能力、内存容量和成本之间提供了不同的权衡关系 (Table 2)。尽管最近的强化学习 (RL) 系统 [17, 59, 69] 倡导将生成与训练在物理上解耦——将回放分配给成本效益高、带宽优化的 GPU (例如 NVIDIA H20), 而将训练分配给计算优化的设备 (例如 H800) ——但我们的分析表明, 这种静态分配对于智能体工作负载来说是不足的。大型语言模型 (LLM) 的生成包含两个资源需求相反的不同阶段: 计算密集型的预填充阶段和内存带宽密集型的解码阶段。在交互轮次较少且推理思维模式的环境中, 大部分运行时间都消耗在解码上 (解码密集型); 相反, 在交互轮次较多的环境中, 预填充阶段占主导地位, 并需要高计算吞吐量 (预填充密集型)。在我们的生产集群中, 智能体强化学习任务呈现出明显的双峰分布, 要么是少量的交互轮次 (<5), 要么是大量的交互轮次 (>10)。

为了量化这种发散, 我们使用 Qwen3-8B/32K 在十次迭代中运行了一个预填充密集型任务 (FrozenLake) 和一个解码密集型任务 (GEM-Math), 并启用了前缀缓存。为了进行成本相当比较, 我们在两种不同的硬件配置上执行工作负载: 一种是 6 块 H20 GPU, 另一种是 2 块 H800 GPU。如 Figure 4a 所示, 计算密集型的 H800 在 FrozenLake 任务上表现优于 H20, 将端到端的推理时间降低至低至 $0.53\times$ 。相反, 在 GEM-Math (Figure 4b) 任务中, H20 出色的内存带宽加速了解码阶段, 使推理时间降至 H800 的 $0.49\times-0.79\times$ 。这些结果否定了“生成过程始终受带宽限制”的教条性假设。相反, 要最大化吞吐量, 需要动态地将任务映射到最适合其特性的硬件上。



(a) 环境时间分布。 (b) 批量环境交互。

图 5: 环境交互分析: (a) 环境初始化 (env.reset) 和环境步进 (env.step) 所用时间 (对数尺度) 的累积分布函数。 (b) 展示长尾环境如何影响批量环境交互下的多轮回放。

R1: 大型语言模型生成的最优资源分配应将工作负载特征 (例如, 预填充-与解码-密集型、批量大小) 与硬件能力相匹配。

长尾环境执行。我们在生产环境中运行数千个并发环境的经验表明, 资源隔离且无干扰是至关重要的。资源共享可能导致严重问题, 例如并发磁盘 I/O 耗尽共享配额, 进而引发级联故障。因此, 我们采用专用的 Kubernetes 簇来管理和隔离每个容器化环境。

大量的先验系统 [14, 20, 69] 分析了大语言模型生成的长尾行为。Figure 5a 展示了 env.reset 和 env.step 操作延迟中类似的长尾分布, 其中 env.reset 的影响尤为显著。在我们的内部生产部署中, env.reset 的长尾延迟可达数百秒, 主要由两个因子导致: (1) 网络竞争, 即数百个环境工作结点同时获取 Docker 镜像会饱和网络链路并压垮镜像注册服务, 以及 (2) 主机结点上的计算与 I/O 竞争, 即启动新容器会消耗大量 CPU 与磁盘资源。

长尾环境会成为拖后腿的因素, 从而延迟端到端的推演延迟。具体而言, 智能体大模型会与多个环境*同时*交互。由于潜在的大模型引擎以批量方式执行请求, 因此将智能体大模型与环境的交互也进行批量处理是自然的选择。Figure 5b 展示了这种批处理执行模式, 其中长尾环境行为可能导致端到端推演延迟显著增加。快速环境必须等待最慢的环境完成, 才能进入下一步生成阶段。

我们的性能分析 (Figure 3) 表明, 相比于理想执行情况, 批量环境交互会使推演时间增加高达 21.3%, 这一开销会随着环境故障率的上升而进一步加剧。

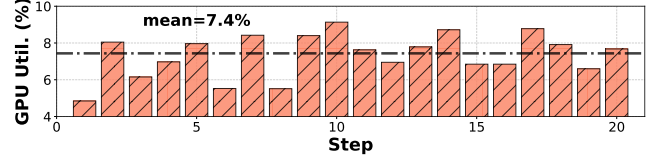


图 6: 将本地 GPU 专门用于奖励计算时资源利用效率低下。

表 3: 从训练簇到推理簇通过 TCP 和 RDMA 的传输开销。

Model	Size (GB)	TCP (s)	RDMA (s)	Speedup
Qwen3-8B	15.26	6.911	5.466	1.264×
Qwen3-14B	27.51	14.437	5.817	2.482×
Qwen3-32B	61.02	29.649	9.442	3.140×

R2: 环境执行, 包括 env.reset 和 env.step, 容易出现极端的长尾延迟。为防止慢速任务阻塞流水线, 系统必须放弃批量环境交互, 转而采用细粒度的异步环境管理。

无状态奖励计算。奖励阶段紧随采样阶段之后, 大多数奖励计算可以实现为无状态函数。对于轻量级的基于代码和基于规则的奖励, 资源需求相对较小, 因此自然可以将这些工作负载卸载到无服务器平台, 以实现可扩展且低延迟的计算。基于大模型的奖励计算需要密集的 GPU 支持, 与采样工作负载竞争 GPU 资源。一种常见做法是为奖励大模型预留专用 GPU。例如, 在我们的 Qwen3-8B/32K SWE-bench 实验中, 批量大小为 128 时, 我们分配 4 块 H800 GPU 给专用的 7B 奖励大模型, 28 块 H800 GPU 用于采样。然而, 奖励 GPU 在整个训练步骤中的平均利用率仅为 7.4% (Figure 6)。由于奖励大模型的参数在训练过程中保持不变, 它可以被视为一个无状态函数, 这使得无服务器部署 [13, 66, 67] 成为提高资源利用率的一个有前景的方法。

R3: 奖励工作者是无状态的, 通常利用率较低, 因此非常适合采用无服务器部署以提高整体资源效率。

3.2 级间通信

除了计算之外, 阶段间通信在 Agentic 强化学习训练中扮演着关键角色, 包含两种不同类型的通信: 对稳定性至关重要的小数据包轨迹传输, 以及对带宽要求较高的大数据量权重更新。

稳定性关键的轨迹传输。与整体训练时间相比, 轨迹数据的传输开销较小。特别是, 由于网络延迟, 频繁的环境交互会导致显著的延迟。

境交互更有可能成为性能瓶颈。在实际应用中，环境交互优先考虑网络稳定性而非网络带宽。环境交互与大语言模型生成之间的异步执行是防止成为这一瓶颈的有效方法。

Bandwidth-Intensive Weight Update. 训练过程中，智能体大模型周期性地更新其权重，这些权重随后必须与回放阶段进行同步。这种权重同步是跨阶段通信开销的主要来源。我们使用 Mooncake [41] 在 TCP (200 Gbps 以太网) 和 RDMA (400 Gbps InfiniBand) 上测量了在训练集群与推理集群之间同步模型参数的端到端传输成本，并在 Table 3 中报告了结果。与 TCP 相比，RDMA 提供了更高的带宽和更低的通信开销。在同步强化学习训练中，回放阶段只能在最新的智能体大模型权重完成同步后才能继续。因此，通过低带宽链路传输权重所产生的高昂成本会增加端到端训练时间，从而削弱了解耦训练的速度提升效果 (Figure 2-左)。

在异步训练 (Figure 2-Right) 中，训练阶段和采样阶段在分离的 GPU 上并行执行。尽管这会引入数据过时效问题，但许多先验工作 [12, 20, 33] 经验性地观察到，即使在高数据过时的情况下，异步训练仍能保持模型质量。鉴于采样阶段占主导地位的开销，异步训练能够有效隐藏训练和权重同步的开销，从而减少端到端训练延迟。

R4: 期望采用异步强化学习后训练以提高训练效率，尤其是在分离式架构中。

4 设计原则

我们提出了设计原则，以满足在分离式基础设施上进行多任务智能体训练的上述需求。

4.1 P1: 硬件亲和性工作负载映射

我们的首要原则是根据工作负载的硬件亲和性将其映射到相应资源，以满足需求 R1 (如 §3.1 框中所示)。用户可在阶段和单个轨迹两个粒度级别上定义一组硬件类型。在阶段层面，训练阶段可运行于 GPU 上，而环境则可在 CPU 上运行。在轨迹层面，该原则可实现更精细的优化。例如，在预填充阶段，生成过程主要由预填充主导且未触及内存限制的轨迹可被路由至 H800 GPU，而以解码为主导的轨迹则可调度至 H20 GPU (§3.1)。类似地，在奖励阶段，奖励大模型可在 GPU 上运行，而代

码沙箱执行则可在 CPU 上运行。这种细粒度的映射实现了效率最大化。

4.2 P2: 细粒度异步执行

为了符合 R2 和 R4，我们引入第二个原则：通过细粒度的异步性来解决解耦智能体训练中的多样化通信模式。特别是，应实现以下设计。

轨迹级回放调度. 传统方法 [50] 以批量方式执行大语言模型生成、环境交互和奖励计算，导致大量资源浪费和较长的回放延迟 (§3.2)。相比之下，轨迹级调度构建了一个连续的流水线，使得一个轨迹的大语言模型生成与另一个轨迹的环境交互以及第三个轨迹的奖励计算重叠进行。这种流水线式执行减少了资源空闲时间，并隐藏了环境通信延迟，提升了对环境不稳定性的影响的鲁棒性。

受控的异步训练同步. 异步强化学习训练将采样与训练阶段解耦，使两者能够并行进行。采样阶段持续生成轨迹，而训练阶段则消耗这些轨迹，并周期性地将更新后的模型权重同步给推理工作者。我们提出的受控同步机制显式地控制已完成轨迹的消费速率。这种控制使得实践者可以灵活配置异步边界 (如 §5.3 中定义)，以在训练稳定性、系统吞吐量和模型权重同步开销之间取得平衡。

4.3 P3: 状态感知计算

第三个原则满足 R3: 我们执行状态感知计算。无状态系统组件的输出仅取决于其输入，使得每次执行相互独立，因此非常适合在无服务器平台上进行最优化。

Rollout: 大模型生成. 生产系统通常采用服务器架构部署大模型生成，以实现低延迟和高通量。最近，Tinker [29] 探索了用于强化学习回放的弹性服务，但其设计围绕 LoRA [23] 展开，而全参数训练对于生产级大模型更为迫切。

Rollout: 环境. 该阶段本质上是状态相关的，因为环境 (例如代码工作区、网页浏览器或游戏) 会随着每个动作而更新。轨迹中的所有动作必须路由到同一个持久化的环境实例，这需要会话保持。

无状态奖励. 一个奖励工作进程接收一条轨迹作为输入，并生成一个标量值，而不保留对过去评估的记忆。这一特性使其非常适合无服务器计算模型，能够实现共享的多租户奖励即服务，可弹性地从零扩展到任意规模，从而最大化资源利用率。


```

1 import rollart.distributed as rdist
2 from rdist.worker import ActorTrainCls, ActorGenCls,
   RewardCls
3 from rdist import ResourceManager as RM
4
5 # 1. Single Controller Example
6 class MyActorTrain(ActorTrainCls):
7     @rdist.register(mode="execute_all")
8     def compute_gradients(self, input_tensor):
9         ...
10
11 # 2. Define actor_gen on heterogeneous GPUs
12 # 2.1 heterogeneous GPU allocation.
13 gen_rm=RM( {"H800": list(range(0, 8))},
14            {"H20", list(range(8, 32))})
15 # 2.2 hardware affinity mapping
16 class HeteroActorGen(ActorGenCls):
17     @rdist.hw_mapping(
18         hw_affinity={"FrozenLake": "H800", "default": "H20"}
19     )
20     def generate(self, input_ids: List[int],
21                 tag_name: str="default"):
22         return self.model.process(prompt)
23
24 # 3. Define a serverless reward computation func
25 class ServerlessRewardWorker(RewardCls):
26     @rdist.register_serverless(
27         attribute='reward_proxy',
28         serverless_url='fc://xxx.xxx')
29     def compute_rewards(self, traj: list):
30         prompt = f"Evaluate the trajectory:{traj}"
31         return ray.get(self.reward_proxy(prompt))

```

Listing 1: The declarative programming model of RollArt.

训练。该阶段本质上是状态相关的，需要专用的 GPU 簇。

5 编程与计算模型

我们引入了 RollArt，该模型通过以下编程与计算模型实现了上述设计原则。

5.1 声明式编程模型

在运行时，工作者是拥有资源的极小点。Listing 1 展示了在工作者函数级别定义的编程模型，使运行时能够对资源使用情况有细粒度的可见性。具体细节如下。**单控制器。**该编程模型在工业强化学习框架中被广泛采用 [50, 72]，因为它简化了智能体强化学习流水线的构建。RollArt 采用此模型，并通过 register 装饰器实现。当执行模式设置为 execute_all (Listing 1 中第 7-8 行) 时，运行时会将输入广播至所有训练者智能体，并在每

个智能体上调用 compute_gradients。RollArt 管理跨智能体的分布式计算与通信，而用户则在每个执行函数内部实现计算逻辑，并通过调用不同类型的智能体的执行函数来组合智能体训练流水线。

硬件亲和性映射 (原则 1)。RollArt 通过基于字典的资源规范支持异构资源组的分配 (第 13 - 14 行)。资源管理器的详细说明见 §6。我们使用 HeteroActorGen 类中的 generate 函数 (第 16 - 22 行) 来说明如何通过 hw_mapping 装饰器 (第 17 - 19 行) 为工作方法声明细粒度的、具有亲和性的硬件映射。具体而言，标记为 FrozenLake 的大语言模型生成工作负载会被高优先级地路由至计算优化型 GPU，而其他工作负载则被高优先级地路由至带宽优化型 GPU。tag_name 参数 (第 21 行) 作为动态路由提示，使得每个大语言模型生成请求能够自动分发至绑定到相应硬件的工作节点。该路由机制确保每条轨迹生成都能以高效的资源亲和性执行。

无服务器注册 (原则 3)。RollArt 暴露 register_serverless (第 26-28 行) 以声明执行方法的状态性。此装饰器允许运行时通过指定的 serverless_url 在无服务器工作节点池上将 compute_rewards (第 29 行) 作为纯函数调用。无服务器平台自动管理资源分配、缩放和请求路由，以实现高资源效率。

5.2 关键抽象: Cluster

上述编程模型简化了 Agentic RL 训练的开发。接下来我们讨论如何通过我们的 Cluster 抽象 (在 Listing 2 中) 来实现。具体而言，该抽象被实现为一个控制器，用于管理分布式工作者在 RL 各阶段的执行。

Distributed Workers. Worker 是 RollArt 中的基本执行单元。作为基类，它可以被具体化为不同角色 (例如，训练、生成) 所需的各类计算单元，以满足各个阶段的需求。Cluster 根据指定的 worker_cls 实例化一组 Worker，并使用资源管理器为每个 Worker 分配资源，同时为其标记资源类型，以便进行硬件亲和性映射 (Listing 2 中第 3 行)。资源管理器拥有并管理一组资源。

Invocation Proxy. _bind_worker_method 函数将指定 worker_cls 的每个方法绑定到 Cluster 实例，使 Cluster 能够作为对应工人簇的代理 (第 4 行)。例如，如果 worker_cls 定义了一个方法 compute_gradients，用户可以直接调用 Cluster.compute_gradients。

我们现在描述支撑我们编程模型的计算模型。首先，

```

1 class Cluster:
2     def __init__(self, res_manager, worker_cls):
3         self._create_worker(worker_cls, res_manager)
4         self._bind_worker_method()
5
6     def execute_all(self, method_name, *args, **kwargs):
7         result = []
8         for worker in self.workers:
9             rcall = getattr(worker, method_name)
10            result.append(rcall(*args, **kwargs))
11        return ray.get(result)
12
13    def hw_mapping(self, hw_affinity, tag_name, *args):
14        hw_type = hw_affinity.get(tag_name)
15        new_workers = []
16        for worker in self.workers:
17            if worker.resource_type == hw_type:
18                new_workers.append(worker)
19        # route requests to new_workers next
20
21    def register_serverless(self, attr, url, *args):
22        # define a call_fc to call serverless url
23        for worker in self.workers:
24            setattr(worker, attr, call_fc)
25        # perform execute_all logic next

```

Listing 2: The simplified implementation and of Cluster and its running example in RollArt.

为了实现单控制器机制，当用户调用带有 `register` 装饰器的方法时，RollArt 进入 `execute_all` 方法，该方法在所有工作节点上调用目标方法并聚合其结果（第 6-11 行）。其次，对于带有 `hw_mapping` 装饰器的方法，RollArt 检查 `tag_name` 参数，筛选出绑定到首选资源类型的工作者，并在这些选定的工作者上执行该方法（第 13 - 19 行）。第三，对于带有 `register_serverless` 装饰器的方法，奖励代理 (`attr`) 被替换为已注册的无服务器 URL，因此奖励计算由无服务器函数完成（第 21 - 25 行）。接下来我们将讨论预填充 - 解码分离以及异步跨簇权重更新。

预填充 - 解码分离。许多系统 [40,68] 将预填充和解码在异构 GPU 之间进行分离。在滚动执行期间，Cluster 允许分布式工作节点利用这些引擎实现这种分离。然而，当前的实际部署仍需要手动配置预填充和解码实例，这很容易导致负载不平衡。因此，我们将其留作未来的工作。

异步跨簇权重更新。由硬件亲和性驱动的分离架构需要在回放（例如，H20）和训练（例如，H800）簇之间进行跨簇通信，这些簇可能物理上相隔较远，仅通过低带宽以太网链路连接。这种缓慢的互联可能成为为单一、统一的高带宽网络设计的框架中权重更新的瓶颈 [50]。RollArt 通过使用 Mooncake 实现异步权重更新引擎来

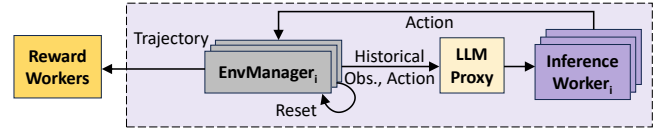


图 7: 轨迹级展开概述。

解决此问题 [41]。在我们的设计中，训练工作节点（位于 H800 簇上）会异步地将权重发布到 Mooncake 存储中，而无需等待并发的回放阶段完成；同时，推理工作节点（位于 H20 簇上）可按需获取这些权重。随后，我们沿用现有的方法 [50] 在簇内执行模型更新。通过有效将跨簇通信开销隐藏在正在进行的轨迹背后，异步跨簇权重更新能够降低端到端训练的开销。

5.3 异步 workflow

我们描述了 RollArt 中实现细粒度异步的异步 workflow (原理 2)。在流水线内部，大语言模型生成与环境执行重叠进行；在各阶段之间，流水线与奖励计算及训练过程重叠进行。

LLMProxy: Trajectory-Level LLM Generation. LLM-Proxy 是一个网关，它将 LLM 服务客户端与潜在的服务实例解耦。它能够智能地在内部 LLM 推理工作器集群中调度请求。每个工作器都围绕一个命令驱动的事件环构建，用于管理推理引擎（例如，vLLM [28], SGLang [45]）。

该环以非阻塞方式持续运行，包含两个组件：(1) 分步命令处理。该环持续轮询来自 LLMProxy 的命令，ADD 用于将新请求入队，ABORT 用于取消现有请求。当无待处理命令时，它通过为一批请求执行单个解码或预填充步骤来推进推理引擎，从而保持高 GPU 利用率。这种设计确保添加或中止正在进行的轨迹不会阻塞整个 LLM 生成过程。(2) 后处理。当 LLM 引擎在完成一定数量的预填充/解码步骤后结束一个请求时，该环会立即调用预先注册的回调函数。该回调对输出进行后处理，并将结果返回给原始客户端（例如，EnvManager）。这使得每个轨迹在 LLM 生成完成后即可立即执行环境交互，而无需等待迟缓的请求。两者共同实现了轨迹级别的 LLM 生成。

EnvManager: Trajectory-level Environment Interaction. 每个 EnvManager 是一个轻量级控制器，用于管理单个环境的生命周期以收集轨迹，如 Figure 7 所示。它通过 reset 进行环境初始化后，进入一个独立的事件环，协调环境实例与共享的 LLMProxy 之间的交互。在此环中，

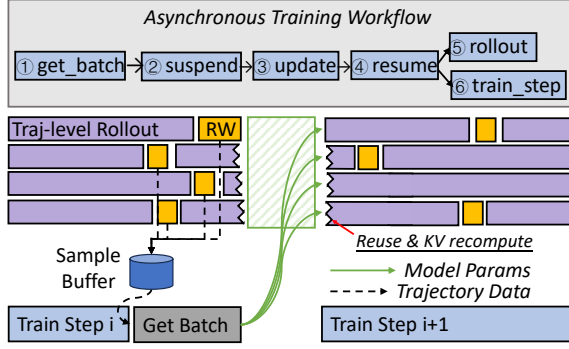


图 8: 异步训练工作流。

EnvManager 维护一个 (观测, 动作) 对列表, 用于构建轨迹。具体而言, 它将历史的 (观测, 动作) 序列作为输入提供给 LLMProxy, 以获取下一个动作, 通过 step 将该动作应用于环境, 并记录产生的观测结果。

在实际应用中, RollArt 同时启动多个 EnvManager 实例, 每个 EnvManager 仅生成一条轨迹, 而非批量执行环境交互 (如 Figure 5b 所示)。因此, 长尾环境工作节点不会延迟其他工作节点的执行。结合轨迹级别的 LLM 生成, RollArt 可以以细粒度的方式重叠 LLM 生成与环境交互。此外, 在完成一条轨迹后, EnvManager 会立即通过无服务器 API 调用奖励计算函数作为非阻塞任务, 从而让奖励计算与正在进行的采样过程重叠。总体而言, 这种基于轨迹的采样管理实现了高度的并行性, 以最大化吞吐量。

最优化: 冗余环境滚动。 LLMProxy 和 EnvManager 的轨迹级设计实现了我们称之为 冗余环境滚动 的优化技术。该技术允许用户启动比与智能体大语言模型交互所必需的更多的环境实例。一旦收集到目标数量的轨迹, 即可终止甚至中止任何正在进行的滚动。由于滚动在轨迹级别进行管理, 慢速环境不会阻塞或延迟快速环境。这有效缓解了“缓慢失败”和“停止失败”的环境问题。我们在 §7.4 中的实证分析表明, 该技术提升了滚动效率。此外, §8 中的大规模部署也从中受益, 避免了环境故障的影响。

Asynchronous Training Workflow. Figure 8 展示了 RollArt 如何协调异步训练工作流。在滚动阶段, 多个 EnvManager 独立运行, 持续生成轨迹并将其入队到 SampleBuffer。大模型推理和训练工作器在分离的 GPU 上运行, 实现了分离的训练。

训练工作流周期性地运行权重同步协议。它首先调用一个阻塞的 get_batch 调用, 从 SampleBuffer 中获取

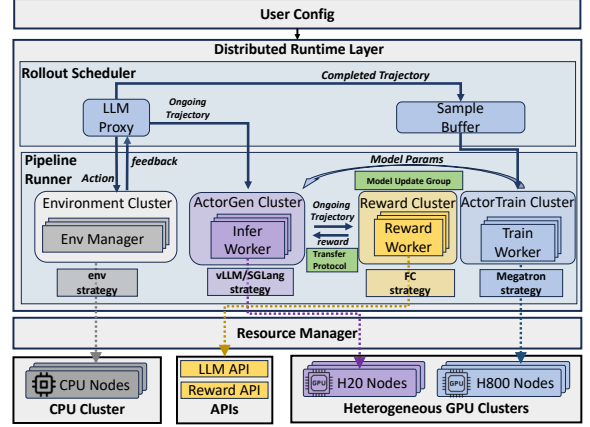


图 9: RollArt 的系统架构

一批轨迹。随后发出 suspend 命令以暂停轨迹收集, 通过获取最新的 LLM 并将其权重广播给所有推理工作节点来执行 model_update。这一过程通过 §5.2 中详述的非阻塞、跨簇通信最优化加速。接着, 它发出 resume 命令, 使用更新后的模型继续进行轨迹级别的滚动推演。上一迭代中未完成的轨迹也会在当前迭代中被重用, 通过 KV 缓存重新计算实现。最后, 它在获取的数据上执行 train_step。在异步训练模式下, 训练阶段与推演阶段重叠, RollArt 通过异步边界控制轨迹的过时程度。

Asynchronous Bound. 在异步训练中, 轨迹生成可以被中断并在更新后的智能体大语言模型 (LLM) 下恢复。因此, 单个批量的轨迹可能由多个版本的 LLM 生成。这种延迟会导致高方差并损害训练稳定性。AReL [12] 通过约束每个批量内样本的新鲜度平均值来保持模型质量。相比之下, RollArt 引入了一个 **异步约束 α** , 以在每条轨迹层面调节新鲜度, 并主动管理异步工作流。该约束定义为: 当前智能体 LLM 与启动该轨迹生成的版本之间的最大允许版本号差距。如果智能体 LLM 已推进到版本 n , 那么 SampleBuffer 中的任何轨迹都必须由不超过版本 $(n - \alpha)$ 的版本发起。违反此约束的轨迹将被终止。我们的实证研究 (§7.2) 表明, 将异步约束设为 1 能够在训练速度和稳定性之间取得平衡。

6 系统架构

我们用大约 6 万行 Python 代码实现了 RollArt。Figure 9 展示了 RollArt 的架构, 包括分布式运行时层和资源管理器。RollArt 解析用户配置, 并在分布式运行时之上构建训练流水线。运行时包含一个回滚调度器, 用于管理回滚执行工作流, 以及一个 Cluster 运行时, 负

责分布式计算与通信。资源管理器从共享池中为工作者分配异构资源。

Rollout 调度器。它通过管理每条轨迹的生命周期来控制 rollout 阶段。LLMProxy 与多个环境管理器交互，并将正在进行的轨迹路由到合适的推理工作者。SampleBuffer 会添加已完成的轨迹，以供后续模型训练使用。

流水线运行器。流水线运行器由多个簇组成，这些簇在智能体强化学习训练中承担不同角色（例如，奖励、环境）。每个簇通过选定的策略（例如，vLLM [28]、Megatron [52]、函数计算 [3]），协调一组工作者执行特定类型的分布式计算（例如，训练、生成、环境交互）。簇之间通过 Ray 的 ObjRef [38] 交换轨迹，该机制支持延迟实例化，并通过传递对象引用而非数据副本，隐藏了数据传输开销。对于模型权重同步，每个簇提供 model_update_group 接口，利用高吞吐量传输引擎（如 NCCL [39] 和 Mooncake [41]），通过 NVLink、InfiniBand 和以太网高效同步权重。

资源管理器。资源管理器使用持久化元数据存储来维护关键状态的实时视图，包括分布式运行时状态、集群资源可用性以及服务 API 可用性。当接收到工作进程部署请求时，它首先查询元数据存储以验证请求，然后将请求的资源或外部服务绑定到工作进程中。

7 性能评估

在本节中，我们展示了端到端评估 (§7.2) 以及对三个设计原则的分析 (§7.3 – 7.5)。

7.1 评估设置

模型与任务。我们在多样化的智能体任务混合数据上训练 Qwen3 [62] 大语言模型家族 (8B – 32B) (Table 1)。所有模型均配置为最大上下文长度为 32K token。由于 SWE-bench 智能体任务的难度，我们仅在 Qwen3-32B 上进行训练。我们采用一个 7B 参数的奖励大语言模型来验证数学任务的推理过程。

训练配置。我们使用 GRPO 算法 [21]，训练批量大小为 512，组大小为 8，并在所有任务间采用均匀采样比例。对于异步训练，我们为训练阶段分配 32 块 H800 GPU，其余的 H20 和 H800 GPU 用于生成。在生成过程中，Qwen3-8B/14B/32B 的张量并行度分别设置为 1、2 和

4，并调整训练并行度以最大化吞吐量。我们使用 vLLM 0.8.4 运行生成，使用 Megatron v0.12.2 进行训练，生成过程中启用前缀缓存和 CUDA 图。

权重更新引擎。我们使用 NCCL [39] v2.26.5 执行簇内权重更新，以及 Mooncake v0.3.7 [41] 存储服务器实现簇间通信。

硬件。RollArt 部署在拥有 96 个 GPU 的 H800 簇和拥有 32 个 GPU 的 H20 簇上。在每个簇内，GPU 结点通过 400 Gbps InfiniBand 连接，而跨簇通信则使用 200 Gbps 以太网网络。我们使用专用的 CPU 簇运行 SWE-bench，另一个用于其余环境。奖励工作者在我们的内部无服务器平台上运行。未特别说明时，所有实验均使用 128 个 GPU。

基准。由于现有开源系统均不支持我们完整的智能体任务谱系，我们采用 veRL (veRL) 的同步强化学习实现作为基准。为增强该基准，我们额外启用了异步奖励计算、异步环境交互以及奖励即服务 (Reward-as-a-Service)，并将其记为 veRL+。我们还与 StreamRL [69] 进行对比，并启用一次性训练范式 [36] (见 Figure 2-左)，该范式通过消耗前一步生成的轨迹来并行化采样与训练。veRL+ 中所用的优化同样应用于 StreamRL。我们在训练阶段使用 Megatron [52]，其不支持异构 GPU 配置。StreamRL 在采样过程中同样不支持异构 GPU。为简化起见，我们将两个基准均运行在 128 张 H800 GPU 上；因此，RollArt 的每 GPU 小时成本约为基准的 83%。

指标。我们将端到端延迟定义为五个迭代的平均步时间。吞吐量为全局批量中提示和响应 token 的总数除以步时间 [51]。我们报告所有任务上的平均验证得分。

7.2 端到端评估

模型收敛。我们每十次迭代测量一次得分，并在 Figure 10a 中报告达到目标得分 0.85 所需的时间。StreamRL 通过重叠训练和回放，实现了相对于 veRL+ 基准 1.52× 的端到端延迟加速。RollArt 通过启动更多回放并跨迭代复用部分生成的轨迹，保持了回放 GPU 的饱和状态。因此，在边界值为 1 的情况下，异步方法相较于 veRL+ 和 StreamRL 分别实现了 2.05× 和 1.35× 的步时间减少。尽管边界值为 2 的异步配置在初始阶段具有更快的收敛速度，但在后期阶段其达到目标得分所需时间略差于边界值为 1 的情况。总体而言，不同边界值仍能提供令人满意的收敛性能。

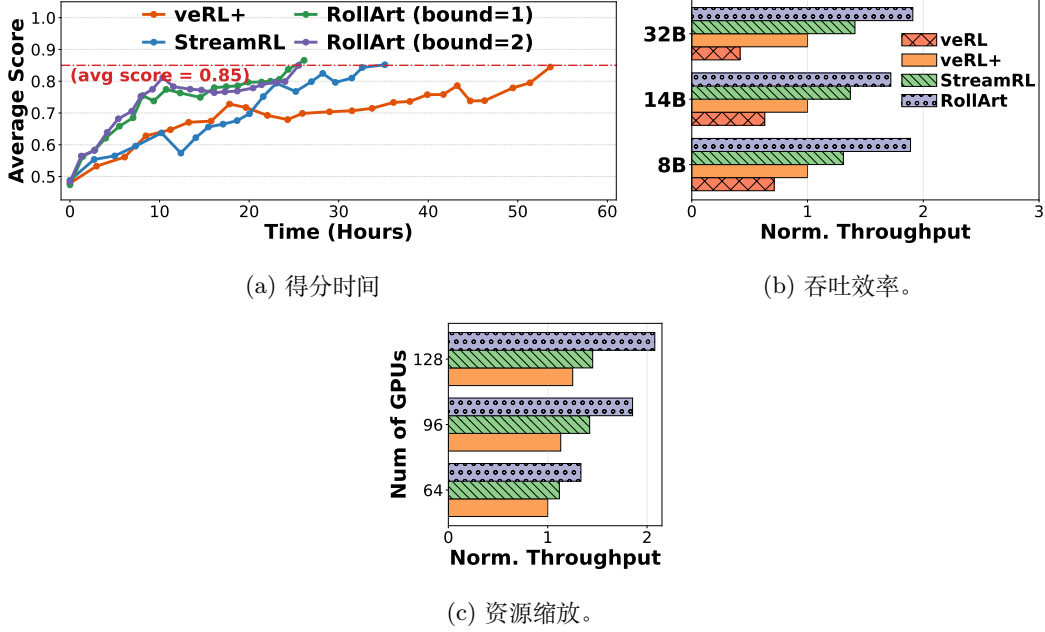


图 10: (a) Qwen3-32B 的端到端 时延得分; (b) 不同方法在各类大模型上的规范化的吞吐量; (c) Qwen3-14B 在不同数量 H800 GPU 下的规范化的吞吐量。

吞吐效率。我们在 Figure 10b 中展示了不同方法的吞吐效率。我们将吞吐结果归一化至 veRL+ 基准。对于 RollArt，我们将其上限设为 1。包括异步奖励和环境交互以及无服务器奖励工作者在内的优化技术可使吞吐量提升 1.40-2.40 $\times$ 。通过重叠采样与训练阶段，我们可以看到 StreamRL 实现了 1.31-1.47 $\times$ 的吞吐量提升。通过细粒度异步性，RollArt 相较于同步基准可提供 2.65-4.58 $\times$ 的吞吐量优势。总体而言，RollArt 在吞吐效率方面展现出显著优势。

资源缩放。我们进一步评估了 RollArt 的资源缩放性能。具体而言，我们在 128 张 H800 GPU 上进行实验，并针对对不同方法将用于采样的 GPU 数量从 64 增加到 128。Figure 10c 报告了相对于在 64 张 H800 GPU 上运行的 veRL+ 基准的规范化的吞吐量。随着分配的 GPU 数量增加，veRL+ 和 StreamRL 的边际吞吐量提升逐渐减弱。然而，RollArt 中的异步 RL 训练仍持续带来 1.33-2.08 $\times$ 的吞吐量改进，展现出更优的资源缩放效率。

7.3 硬件亲和性分析

训练效率。我们在大模型上评估了计算优化和带宽优化硬件的训练效率。为隔离硬件亲和性的影响，我们将训练分配固定为 32 张 H800 GPU，并采用三种资源配置进

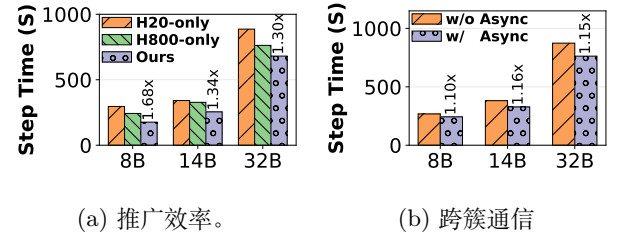
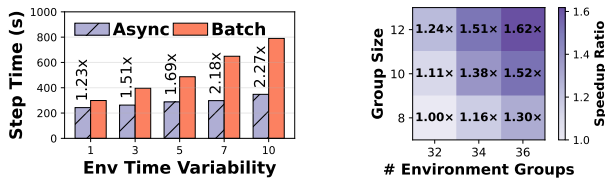


图 11: [原理 1]: (a) 硬件亲和性的效率; (b) 异步跨簇通信的优势。

行推理：72 张 H800 GPU 作为仅 H800 的基准，208 张 H20 GPU 作为仅 H20 的基准，以及一个具有亲和性的设置，包含 64 张 H800 GPU 加 24 张 H20 GPU。在亲和性设置中，数学和游戏导向的智能体任务被优先路由至 H20 GPU。如 Figure 11a 所示，相较于仅使用 H20 的配置，RollArt 在不同大模型规模下实现了 1.30 – 1.68 $\times$ 的每步时间加速；相较于仅使用 H800 的配置，也实现了 1.12 – 1.37 的加速，这是由于更多 H20 GPU 的扫描减少了资源争用，并有利于解码密集型任务。仅使用 H20 的配置表现最差，表明许多智能体任务因频繁的预填充操作而更受益于计算优化的 GPU。总体而言，利用异构硬件在推理中的互补优势对于提升训练效率至关重要。

异步跨簇通信。硬件亲和性引入了跨簇权重更新，其中通过以太网连接的簇会产生较高的通信开销。与 veRL



(a) 环境时间变异性。 (b) 冗余的 Env 发布。

图 12: [原理 2]: 轨迹级环境的益处

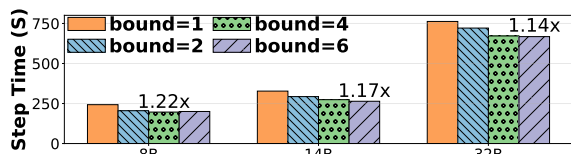


图 13: [原理 2]: 不同异步边界在大模型上的平均步时间对比。

基于 NCCL 的通信方式不同——后者假设所有 GPU 服务器均通过统一高带宽链路连接——在我们的情景中必须显式处理异构互连。Figure 11b 对比了我们异步跨簇通信与 veRL 方法的端到端步骤时间。异步通信技术在大模型上实现了 1.10-1.16 $\times$ 的端到端步骤时间降低，表明跨簇通信开销值得专门优化。

7.4 轨迹级异步性分析

异步环境交互。 轨迹级环境管理实现了异步环境交互。我们运行 Qwen3-8B/32K，并在每轮中注入从均值为 $\mu = 10$ 、标准差 σ （范围为 1 到 10）的高斯分布中采样的额外环境延迟。我们在 Figure 12a 中对比了轨迹级与批量级交互，报告了十次迭代的平均步时。随着延迟方差增大，轨迹级相对于批量级交互的性能提升从 1.23 $\times$ 增长至 2.27 $\times$ ，证明异步环境交互能够维持高通量。

冗余环境回放。 GRPO 暴露了两个关键配置参数：环境组的数量和组的大小。通过启动额外的环境，冗余回放可以容忍环境故障并加速训练。我们在 32 块 H800 GPU 上运行 Qwen3-8B/32K，并在 GEM-math Agentic 任务上调整环境组数量和组大小，报告回放加速比于 Figure 12b。最大加速比达到 1.62 $\times$ ，增加任一参数均带来正向加速。

异步因子上限的影响。 我们将异步因子上限从 1 调整至 6，并在 Figure 13 中报告大模型的平均步时间。增大上限通常会降低因过时导致已完成轨迹被中止的概率，从而在大多数情况下减少步时间。然而，我们观察到效率在某些范围内趋于平稳。不同大模型的最优上限存在差异，与上限为 1 的情况相比，最多可使步时间减少 1.22 $\times$ 。

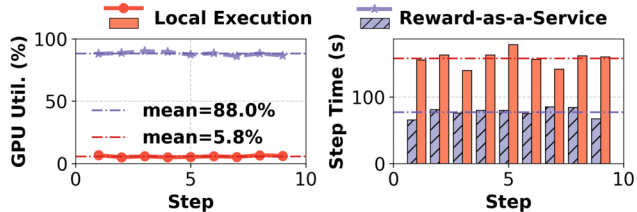


图 14: [原则 3]: 投入本地 GPU 与使用奖励即服务 (Reward-as-a-Service) 之间的比较。

考虑到整体训练性能，这些吞吐量提升并不一定转化为更优的得分时间。实际应用中，将上限设为 1 即可获得令人满意的性能。

7.5 无服务器奖励工作者的优势

我们通过与专用本地 GPU 设置的对比来评估 Reward-as-a-Service。我们在一个 16-GPU 簇上并行运行三个 Agentic 强化学习任务，使用 Qwen3-8B/16K 作为智能体，Qwen2.5-7B 作为奖励语言模型，其中八张 GPU 用于训练。在本地设置中，四张 GPU 被预留用于奖励语言模型。Figure 14 展示了批量大小为 84 时的 GPU 利用率和每步的采样时间，其中采样时间包含异步奖励计算。无服务器平台由三个任务共享，并支持自动缩放，使平均 GPU 利用率从 6% 提升至 88%。由于无需为奖励计算保留热备 GPU，更多 GPU 可用于采样，因此平均采样时间从 158 秒降低至 77 秒，体现了状态感知计算的优势。

8 生产中的 RollArt

在过去六个月中，成千上万的 Agentic RL 任务使用 RollArt 进行后训练。为了进一步展示其可扩展性和鲁棒性，我们分享了在拥有超过 3,000 块 GPU 的大规模生产簇上的使用经验。

Workload Characterization 我们使用 RollArt 在内部数据集上对一个 MoE 大语言模型（数千亿参数）进行了训练，用于数学和软件工程等智能体任务。最大提示长度和响应长度分别为 12k 和 46k。每项任务的平均轮次从 1 到 48 不等 (Figure 15a)，证实了在智能体强化学习训练中，联合预填充密集型与解码密集型任务共存。大量轮次要求环境具有高度稳定性以及快速的预填充计算能力。

此任务采用异步训练，训练与生成 GPU 的比例为

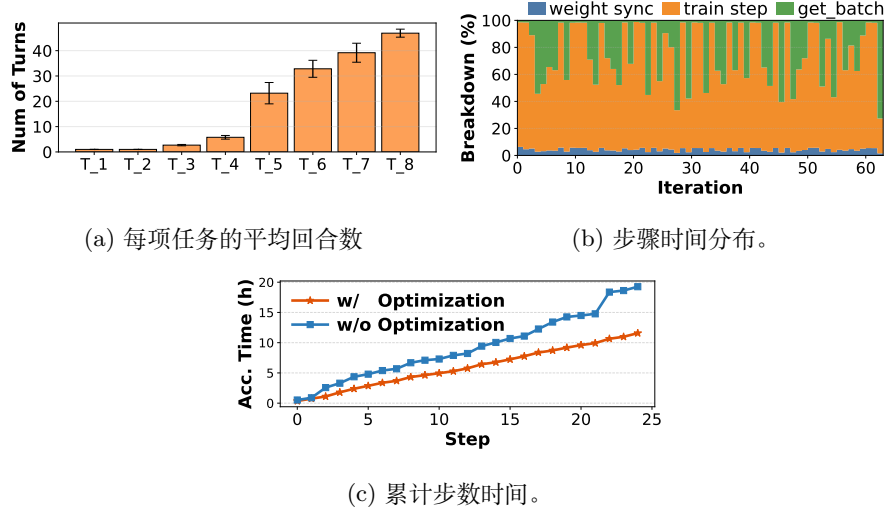


图 15: 生产级 Agentic 强化学习工作负载特征分析。

1:5。为平衡梯度稳定性与回滚效率，使用一个异步边界值为 1，导致最大迭代时间为 1.5 小时（参见 Figure 15b 中的迭代时间分布）。主要瓶颈是训练阶段完成对数概率和梯度计算后所阻塞的 `get_batch` 调用。该调用需等待 `SampleBuffer` 中积累足够的轨迹以达到批量大小。这会消耗最多达 62% 的迭代时间，并造成 GPU 空闲，若消除该瓶颈，理论上可将端到端训练时间减少 22% (Figure 15b)。由于回滚阶段提供的轨迹不足，训练阶段可能陷入停滞，因此必须平衡两个阶段的吞吐量。

Characterization-driven Optimization. 基于工作负载特征分析，我们调整了资源分配比例，并对 MoE 架构的前缀缓存进行了最优化。如 Figure 15c 所示，基于特征分析的最优化在前 25 步中实现了 $1.66\times$ 的端到端加速。此外，我们还为环境和系统鲁棒性提供了专门的优化。

Optimizing Environment Stability. 在 Kubernetes 上运行数千个并发的基于 Docker 的智能体环境，需要进行关键优化以确保稳定性和性能。为应对 `env.reset` 中的网络不稳定和拉取开销问题，我们采用多层缓存架构。第一层是一个内部镜像仓库，作为本地镜像副本，消除了对外部网络的调用。第二层是位于计算结点与该仓库之间的分布式负载均衡缓存，用于高效处理高并发请求。在执行 `env.reset` 操作时，客户端优先从该缓存中获取所需的 Docker 镜像。这一方法显著提升了环境初始化的鲁棒性，使 `env.reset` 的成功率超过 99.99%。

系统韧性. 我们在网络和系统两个层面增强了系统韧性。为减少连接环境时的超时情况，我们采用基于持久

会话的协议，而非基于请求的协议，并引入指数退避重试机制以应对临时的网络问题。我们的解耦架构能够隔离故障，确保某一环境、奖励或推理工作器出现问题时不会影响其他组件。Kubernetes 负责管理环境工作器，而无服务器平台负责管理奖励工作器。当一个推理工作器失败时，首先会在同一块 GPU 上重启；若再次失败，则将其移除，并在健康的工作器上从存储引擎中恢复其轨迹数据。当训练工作器失败时，我们从最新的检查点重新启动。这种设计已被证明具有极高的鲁棒性，在长达一周的训练运行中仅观察到一次故障。

9 相关工作

强化学习后训练系统. 许多系统致力于解决强化学习后训练中的系统挑战。早期的框架 [19, 24, 64] 采用静态、阶段特定的 GPU 划分方式，当各阶段工作负载分布发生变化时，会降低整体利用率。veRL [50] 则采用混合控制器，将多个阶段共置于同一组 GPU 上，以提高硬件效率。后续系统 [6, 7, 12, 17, 20, 31, 47, 49, 69, 70] 探索了多种加速技术，包括推测解码、流水线阶段融合以减少编排开销、异步或解耦训练以隐藏延迟，以及各种形式的资源解聚。RollArt 基于解聚和异步范式，根据硬件亲和性分配工作负载。

Resource Disaggregation. 资源分离的概念最早在早期关于内存的架构工作中被提出 [30]，如今已成为现代系统设计的基石。最近的系统如 LegoOS [46] 和 Mira [16] 将硬件解耦为可独立管理的资源池以提高利用率。这种

模式在大语言模型 (LLM) 服务中十分普遍, 系统通常会将预填充 (prefill) 和解码 (decoding) 阶段分离 [22, 40, 41, 56, 68], 有些甚至进一步将 Transformer 子模块如注意力 (Attention) 和前馈网络 (FFNs) 进行分离 [55, 71]。在强化学习 (RL) 后训练场景中, 一些工作也应用了类似原则, 将训练和回放 (rollout) 阶段分别部署在不同类型的 GPU 上 [59, 69]。与这些方法相比, RollArt 提供了一个更通用且粒度更细的分离模型, 专门针对多任务 Agentic RL 的整个生命周期进行优化。

10 结论

在本文中, 我们设计了一个高效且可扩展的分离式强化学习训练系统。我们介绍了其编程模型、计算模型和系统架构, 所有设计均遵循三个核心原则。我们的微基准测试和宏基准测试表明, RollArt 在生产级簇中显著提升了资源效率、可扩展性和系统韧性。

参考文献

- [1] Introducing openai o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>, 2024.
- [2] Alibaba. Rtp-llm. <https://github.com/alibaba/rtp-llm>. Accessed: 2025-12.
- [3] Alibaba Cloud. Alibaba Cloud Function Compute. <https://www.alibabacloud.com/en/product/function-compute>. Accessed: 2025-12.
- [4] The Kubernetes Authors. Kubernetes, 2025.
- [5] Axon-RL. Gem: Generalist environment for multi-task learning.
- [6] Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. Respec: Towards optimizing speculative decoding in reinforcement learning systems, 2025.
- [7] Rongxin Cheng, Kai Zhou, Xingda Wei, Siyuan Liu, Mingcong Han, Mingjing Ai, Yeju Zhou, Baoquan Zhong, Wencong Xiao, Rong Chen, and Haibo Chen. Fast llm post-training via decoupled and best-of-n speculation, 2025.
- [8] Alibaba Cloud. Qwen model repos, 2025.
- [9] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.
- [10] DeepSeek-AI. Deepseek-v3.2-speciale. <https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Speciale>, 2025. Accessed: 2025-12.
- [11] Farama Foundation. Gymnasium - frozenlake environment. https://gymnasium.farama.org/environments/toy_text/frozen_lake/, 2024. Accessed: 2025-09.
- [12] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025.
- [13] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-Latency serverless inference for large language models. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pages 135–153, Santa Clara, CA, July 2024. USENIX Association.
- [14] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training, 2025.
- [15] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. arXiv preprint arXiv:2401.14196, 2024.

- [16] Zhiyuan Guo, Zijian He, and Yiying Zhang. Mira: A program-behavior-guided far memory system. In *sosp*, 2023.
- [17] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, Huazhong Ji, Wenjie Liu, Yu Huang, Yixiang Zhang, Chenyi Pan, Jing Wang, Xin Huang, Chunsheng Li, and Jianping Wu. Asyncflow: An asynchronous streaming rl framework for efficient llm post-training, 2025.
- [18] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Yicheng Chen, Cunyin Peng, Jinjie GU, and Chenyi Zhuang. Exploring superior function calls via reinforcement learning, 2025.
- [19] Eric Harper, Somshubra Majumdar, Oleksii Kuchaiev, Li Jason, Yang Zhang, Evelina Bakhurina, Vahid Noroozi, Sandeep Subramanian, Koluguri Nithin, Huang Jocelyn, Fei Jia, Jagadeesh Balam, Xuesong Yang, Micha Livne, Yi Dong, Sean Naren, and Boris Ginsburg. NeMo: a toolkit for Conversational AI and Large Language Models, 2025.
- [20] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History rhymes: Accelerating llm reinforcement learning with rhymerrl, 2025.
- [21] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.
- [22] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [24] Jian Hu, Xibin Wu, Weixun Wang, Dehao Zhang, Yu Cao, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.
- [25] Pengcheng Jiang, Jiacheng Lin, Lang Cao, Runchu Tian, SeongKu Kang, Zifeng Wang, Jimeng Sun, and Jiawei Han. Deepretrieval: Hacking real search engines and retrievers with large language models via reinforcement learning. *CoRR*, abs/2503.00223, March 2025.
- [26] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.
- [27] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning, 2025.
- [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [29] Thinking Machines Lab. Tinker, 2025.
- [30] Kevin Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K Reinhardt, and Thomas F Wenisch. Disaggregated memory for expansion and sharing in blade servers. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pages 267–278, 2009.

- [31] Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Anxiang Zeng, and Jinsong Su. Spec-rl: Accelerating on-policy reinforcement learning via speculative rollouts, 2025.
- [32] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners, 2025.
- [33] Han Lu, Zichen Liu, Shaopan Xiong, Yancheng He, Wei Gao, Yanan Wu, Weixun Wang, Jiashun Liu, Yang Li, Haizhou Zhao, Ju Huang, Siran Yang, Xiaoyang Li, Yijia Luo, Zihe Liu, Ling Pan, Junchi Yan, Wei Wang, Wenbo Su, Jiamang Wang, Lin Qu, and Bo Zheng. Part ii: Roll flash – accelerating rlvr and agentic training with asynchrony, 2025.
- [34] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guan-jing Xiong, and Hongsheng Li. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning, 2025.
- [35] Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpav Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. Deepsw: Training a state-of-the-art coding agent from scratch by scaling rl, 2025. Notion Blog.
- [36] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl, 2025. Notion Blog.
- [37] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. Gui-r1 : A generalist r1-style vision-language action model for gui agents, 2025.
- [38] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 561–577, Carlsbad, CA, October 2018. USENIX Association.
- [39] NVIDIA Corporation. NVIDIA Collective Communication Library (NCCL). <https://github.com/NVIDIA/nccl>. Accessed: 2025-12.
- [40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In isca, 2024.
- [41] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Kimi’s kvcache-centric architecture for llm serving. arXiv preprint arXiv:2407.00079, 2024.
- [42] Alibaba Qoder. Qoder: Agentic coding platform for real software, 2025.
- [43] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [44] ByteDance Seed, Yufeng Yuan, Yu Yue, Mingxuan Wang, Xiaochen Zuo, Jiaze Chen, Lin Yan, Wenyan Xu, Chi Zhang, Xin Liu, et al. Seed-thinking-v1. 5: Advancing superb reasoning models with reinforcement learning. arXiv preprint arXiv:2504.13914, 2025.
- [45] SGLang Team. Sglang: Fast serving framework for large language models. <https://github.com/sgl-project/sglang>, 2025. Version 0.4.
- [46] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In osdi, 2018.

- [47] Zelei Shao, Vikranth Srivatsa, Sanjana Srivastava, Qingyang Wu, Alpay Ariyak, Xiaoxia Wu, Ameen Patel, Jue Wang, Percy Liang, Tri Dao, Ce Zhang, Yiyang Zhang, Ben Athiwaratkun, Chenfeng Xu, and Junxiong Wang. Beat the long tail: Distribution-aware speculative decoding for rl training, 2025.
- [48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- [49] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Laminar: A scalable asynchronous rl post-training framework, 2025.
- [50] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. verl: Volcano engine reinforcement learning for llm. <https://github.com/volcengine/verl>, 2024.
- [51] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In ACM EuroSys, 2025.
- [52] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.
- [53] Guijin Son, Hyunwoo Ko, Hoyoung Lee, Yewon Kim, and Seunghyeok Hong. Llm-as-a-judge and reward model: What they can and cannot do, 2024.
- [54] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning, 2025.
- [55] StepFun. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding, 2025.
- [56] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. Déjàvu: Kvcache streaming for fast, fault-tolerant generative llm serving. In icml, 2024.
- [57] Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat, Kyle Montgomery, Aaron Hao, Tianhao Wu, Arnab Balyan, Manan Roongta, Chenguang Wang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. rllm: A framework for post-training language agents, 2025. Notion Blog.
- [58] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang,

- Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2025.
- [59] Jinghui Wang, Shaojie Wang, Yinghan Cui, Xuxing Chen, Chao Wang, Xiaojiang Zhang, Minglei Zhang, Jiarong Zhang, Wenhao Zhuang, Yuchen Cao, Wankang Bao, Haimo Li, Zheng Lin, Huiming Wang, Haoyang Huang, Zongxian Feng, Zizheng Zhan, Ken Deng, Wen Xiang, Huaixi Tang, Kun Wu, Mengtong Li, Mengfei Xie, Junyi Peng, Haotian Zhang, Bin Chen, and Bing Yu. Seamlessflow: A trainer agent isolation rl framework achieving bubble-free pipelines via tag scheduling, 2025.
- [60] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Zihe Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library, 2025.
- [61] Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 28489–28503, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [62] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.
- [63] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023.
- [64] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song,

- and Yuxiong He. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales, 2023.
- [65] Chengyue Yu, Siyuan Lu, Chenyi Zhuang, Dong Wang, Qintong Wu, Zongyue Li, Runsheng Gan, Chunfeng Wang, Siqi Hou, Gaochi Huang, Wenlong Yan, Lifeng Hong, Aohui Xue, Yanfeng Wang, Jinjie Gu, David Tsai, and Tao Lin. Aworld: Orchestrating the training recipe for agentic ai, 2025.
- [66] Minchen Yu, Ao Wang, Dong Chen, Haoxuan Yu, Xiaonan Luo, Zhuohao Li, Wei Wang, Ruichuan Chen, Dapeng Nie, Haoran Yang, and Yu Ding. Torpor: Gpu-enabled serverless computing for low-latency, resource-efficient inference. In 2025 USENIX Annual Technical Conference (USENIX ATC 25), pages 597–612, 2025.
- [67] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. Blitzscale: Fast and live large model autoscaling with $\mathcal{O}(1)$ host caching, 2025.
- [68] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024.
- [69] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation, 2025.
- [70] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. Rlhfuse: Efficient rlhf training for large language models with inter- and intra-stage fusion, 2024.
- [71] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. Megascale-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism, 2025.
- [72] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.